



```
#include<bits/stdc++.h>
using namespace std;
using ll = long long int;
const ld ESP = 1e-10;
const ld PI = acosl(-1);
//随机化
random_device rd;
mt19937 gen(rd());
uniform_int_distribution<> dis(1, 100);
void solve(){
    ...
}
int main(){
    // freopen("zcx.in","r",stdin);
    // freopen("zcx.out","w",stdout);
    int _=1;
    cin>>_;
    while(_--){
        solve();
    }
    cerr<<clock()*1.0/CLOCKS_PER_SEC<<'\\n';//在终端输出运行时间
    return 0;
}
```

编译运行C++

- A.sh 脚本如下

```
chmod +x A.sh
```

chmod 是 change mode（改变权限）的命令。

+x 表示给文件增加“可执行权限”。

A.sh 是你的脚本文件。

执行后，你就可以直接用 `./A.sh` 运行，而不需要 `bash A.sh`

```
./A.sh
```

./ 表示当前目录下执行文件。

系统会读取 `A.sh` 文件内容，根据首行 `#!/bin/sh` 或 `#!/usr/bin/env bash` 选择解释器执行，脚本里的命令按顺序执行。

```

# /bin/sh
g++ -std=c++17 -O2 -Wall A.cpp -o A
./A < in.txt > out.txt
cat out.txt;echo
diff out.txt std.txt

#!/bin/sh
# 脚本用途：编译 A.cpp，运行生成输出 out.txt，并与标准答案 std.txt 做对比
# 适用于算法竞赛或本地测试 C++ 程序
# -----
# 1. 编译 C++ 程序
g++ -std=c++17 -O2 -Wall A.cpp -o A #
# g++          -> GNU C++ 编译器
# -std=c++17   -> 使用 C++17 标准
# -O2          -> 优化等级 O2（提高运行效率，常用于竞赛）
# -Wall        -> 打开所有警告（帮助发现潜在问题）
# A.cpp        -> 源文件
# -o A         -> 指定生成的可执行文件名为 A
# 注：如果编译失败（语法错误、缺少头文件等），脚本会报错并停止
# -----
# 2. 运行可执行程序
./A < in.txt > out.txt #
# ./A          -> 执行当前目录下的可执行文件 A
# < in.txt     -> 将 in.txt 作为标准输入传递给程序
# > out.txt    -> 将程序的标准输出写入 out.txt 文件（覆盖原文件）
# -----
# 3. 查看程序输出
cat out.txt;echo #
# cat out.txt  -> 将 out.txt 的内容打印到终端
# echo         -> 输出一个空行，让终端显示更整齐
# 这样可以快速检查程序输出
# -----
# 4. 对比输出与标准答案
diff out.txt std.txt #
# diff          -> Linux 命令，用于比较两个文件差异
# out.txt       -> 程序生成的输出
# std.txt       -> 标准答案文件
# diff 默认输出具体的差异：
# - 如果文件相同 → diff 不输出内容，退出码为 0

```

```
# - 如果文件不同 → diff 输出差异，退出码为 1
# -----
```

对拍脚本

```
#!/bin/sh

g++ -std=c++17 -O2 -Wall Z.cpp -o Z    #造数据
./Z > in.txt
g++ -std=c++17 -O2 -Wall bl.cpp -o bl  #暴力
./bl < in.txt > std.txt
g++ -std=c++17 -O2 -Wall A.cpp -o A    #当前
./A < in.txt > out.txt
# cat out.txt; echo
# diff out.txt std.txt
```